

REGN NO : FA25-BCS-017

NAME:FAREEHA BATOOL

DATA STRUCTURE LAB ASSIGNMENT NO 01

```
#include <iostream>
#include <cmath>
using namespace std;

const int CAPACITY = 20;

struct ArrayList
{
    int data[CAPACITY];
    int size = 0;
};

// Insert at End
bool insertEnd(ArrayList &list, int value)
{
    if (list.size == CAPACITY)
        return false;

    list.data[list.size] = value;
    list.size++;

    return true;
}

// Insert at Beginning
bool insertAtBeginning(ArrayList &list, int value)
{
    if (list.size == CAPACITY)
        return false;

    for (int i = list.size; i > 0; i--)
    {
        list.data[i] = list.data[i - 1];
    }

    list.data[0] = value;
```

```

        list.size++;

        return true;
    }

// Delete from Position
bool deleteAtPosition(ArrayList &list, int position)
{
    if (position < 0 || position >= list.size)
        return false;

    for (int i = position; i < list.size - 1; i++)
    {
        list.data[i] = list.data[i + 1];
    }

    list.size--;

    return true;
}

// Display List
void displayList(const ArrayList &list)
{
    for (int i = 0; i < list.size; i++)
        cout << list.data[i] << " ";

    cout << endl;
}

int main()
{
    ArrayList list;

    int *ptr = list.data;
    int *minPtr = list.data;
    int *maxPtr = list.data;
    int *medianPtr = list.data;
    int *closestPtr = list.data;

```

```

int sum = 0;
int closestPosition;

double generalAverage;
double specialAverage;
double averageDifference;
double finalScore;

// PART A

insertEnd(list, 18);
insertEnd(list, 7);
insertEnd(list, 45);
insertEnd(list, 11);
insertEnd(list, 36);
insertEnd(list, 17);    // Registration No. last two digits
insertEnd(list, 21);
insertEnd(list, 13);
insertEnd(list, 29);

cout << "Initial ArrayList: ";
displayList(list);

// PART B
// Sum, Minimum, Maximum
ptr = list.data;
minPtr = list.data;
maxPtr = list.data;
sum = 0;

for (int i = 0; i < list.size; i++)
{
    sum = sum + *ptr;

    if (*ptr < *minPtr)
        minPtr = ptr;

    if (*ptr > *maxPtr)

```

```

        maxPtr = ptr;

    ptr++;
}

cout << "Minimum Value: " << *minPtr << endl;
cout << "Maximum Value: " << *maxPtr << endl;
cout << "Sum: " << sum << endl;
// PART C
// Median

int temp[CAPACITY];

// Copy ArrayList into temporary array
for (int i = 0; i < list.size; i++)
{
    temp[i] = list.data[i];
}

// Manual Bubble Sort
for (int i = 0; i < list.size - 1; i++)
{
    for (int j = 0; j < list.size - i - 1; j++)
    {
        if (temp[j] > temp[j + 1])
        {
            int x = temp[j];
            temp[j] = temp[j + 1];
            temp[j + 1] = x;
        }
    }
}

int medianValue = temp[list.size / 2];

// Find median in ORIGINAL ArrayList
medianPtr = list.data;

```

```

for (int i = 0; i < list.size; i++)
{
    if (*medianPtr == medianValue)
        break;

    medianPtr++;
}

cout << "Median Value: " << *medianPtr << endl;

// PART D
// Averages

generalAverage = (double)sum / list.size;

specialAverage =
    (*minPtr + *medianPtr + *maxPtr) / 3.0;

cout << "General Average: " << generalAverage << endl;
cout << "Special Average: " << specialAverage << endl;

// Find Closest Value
ptr = list.data;
closestPtr = list.data;

double difference = fabs(*ptr - specialAverage);

for (int i = 0; i < list.size; i++)
{
    double currentDifference =
        fabs(*ptr - specialAverage);

    if (currentDifference < difference)
    {
        difference = currentDifference;
        closestPtr = ptr;
    }
}

```

```

        ptr++;
    }

    // Pointer Arithmetic
    closestPosition = closestPtr - list.data;

    cout << "Closest Value: " << *closestPtr << endl;
    cout << "Position of Closest Value: "
        << closestPosition << endl;

    // PART E
    // Final Calculations

    averageDifference =
        fabs(generalAverage - specialAverage);

    finalScore =
        fabs(*closestPtr - generalAverage)
        + fabs(*closestPtr - specialAverage)
        + averageDifference;

    cout << "Difference Between Averages: "
        << averageDifference << endl;

    cout << "Final Score: " << finalScore << endl;

    // Delete Closest Value
    closestPosition = closestPtr - list.data;

    deleteAtPosition(list, closestPosition);

    cout << "ArrayList After Deletion: ";
    displayList(list);

    // Round Special Average
    int roundedAverage = round(specialAverage);

```

```

// Insert rounded average at beginning
insertAtBeginning(list, roundedAverage);

cout << "Final ArrayList After Insertion: ";
displayList(list);

return 0;
}

```

```

Initial ArrayList: 18 7 45 11 36 17 21 13 29
Minimum Value: 7
Maximum Value: 45
Sum: 197
Median Value: 18
General Average: 21.8889
Special Average: 23.3333
Closest Value: 21
Position of Closest Value: 6
Difference Between Averages: 1.44444
Final Score: 4.66667
ArrayList After Deletion: 18 7 45 11 36 17 13 29
Final ArrayList After Insertion: 23 18 7 45 11 36 17 13 29

-----
Process exited after 0.3396 seconds with return value 0
Press any key to continue . . . |

```